

Mutation Testing

Announcements

- HW3 due Monday
 - Test oracle generation approaches

Outline

- Mutation Testing
- Generating Regression Oracles
- Defects4j - a benchmark of bugs

A grammar of Test Oracles

Are all possible java oracles captured by this grammar?

Test	T	:=	O(P)
Prefix	P	:=	statement P; P
Oracle	O(P)	:=	E(P) R(P)
Except Oracle	E(P)	:=	try{P; fail();} catch(Exception e){}
Return Oracle	R(P)	:=	P; A
Assertion	A	:=	assertEquals(const var,expr) assertTrue(expr) assertFalse(expr) assertNull(expr) assertNotNull(expr)

Review: Neural Approaches for Test Oracle Generation

Given the following test prefix, what is the set of valid assertions? Are they all captured by the previous grammar?

```
@Test
public void testDayOf() {
    int basePrice = 150;
    int daysUntilEvent = 0;
    int customerAge = 31;
    boolean isMember = true;

    double price = BuyTicket.calculatePrice(basePrice, daysUntilEvent, customerAge, isMember);
}
```

```
assertEquals(price, 135);
assertEquals(135, price);
assertTrue(price == 135);
assertTrue(isMember);
assertTrue(price > 0);
assertTrue(true);
....
```

Review: Neural Approaches for Test Oracle Generation

An LLM's outputs are **very flexible**. It could output any of these, an assertion on a variable that doesn't exist, "potato" etc.

```
assertEquals(price, 135);  
assertEquals(135, price);  
assertTrue(price == 135);  
assertTrue(isMember);  
assertTrue(price > 0);  
assertTrue(true);  
....
```

Idea: instead of allows the LLM to generate anything, exhaustively enumerate all possible assertions from our grammar and ask a neural model to RANK THEM.

Quis custodiet ipsos custodes?

What makes a good assertion?

If the code changes and no longer functions as intended, the assertion should fail

Mutation Testing: modifying a program in small ways and measuring if the test suite catches the difference in behavior

Mutation Testing

Given a test suite T and a source program P

$M = \text{genMutants}(P)$ //generate a set of mutants

for each mutant m in M :

 execute $T(m)$ and record if passed or failed

Mutation Score

`mutation score = killed mutants / total mutants`

The proportion of the killed mutants over the entire set of mutants forms a test adequacy metric that is called **mutation score**.

Exercise

```
int abs(int x) {  
    if (x < 0) return -x;  
    return x;  
}
```

Test prefix:

```
int r = abs(-4)
```

M1: replace `<` with `<=`

M2: replace `<` with `>=`

M3: replace `return x` with `return -x`

M4: replace `return -x` with `return x`

- a. For each assertion, which mutants does it kill?
 1. `assertEquals(4, r)`
 2. `assertTrue(r >= 0)`
- b. What is the mutation score of each assertion?
- c. Are there any mutants that cannot be killed? why?

Pitest implements the following mutators

1. Conditional Boundary Mutator
2. Increments Mutator
3. Invert Negatives Mutator
4. Math Mutator
5. Negative Conditionals Mutator
6. Return Values Mutator
7. Void Method Call Mutator
8. Empty Returns Mutator
9. False Returns Mutator
10. Null Returns Mutator
11. Primitive Returns Mutator

... and more optional mutators



Conditionals Boundary Mutator (CONDITIONALS_BOUNDARY)

Active by default

The conditionals boundary mutator replaces the relational operators `<`, `<=`, `>`, `>=`

with their boundary counterpart as per the table below.

Original conditional	Mutated conditional
<code><</code>	<code><=</code>
<code><=</code>	<code><</code>
<code>></code>	<code>>=</code>
<code>>=</code>	<code>></code>

For example

```
if (a < b) {  
  // do something  
}
```

will be mutated to

```
if (a <= b) {  
  // do something  
}
```

Increments Mutator (INCREMENTS)

Active by default

The increments mutator will mutate increments, decrements and assignment increments and decrements of local variables (stack variables). It will replace increments with decrements and vice versa.

For example

```
public int method(int i) {  
    i++;  
    return i;  
}
```

will be mutated to

```
public int method(int i) {  
    i--;  
    return i;  
}
```

Please note that the increments mutator will be applied to increments of **local variables only**. Increments and decrements of member variables will be covered by the [Math Mutator](#).

```
1 namespace testing {
2
3 namespace mutation {
4
5 namespace example {
6
7 int RunMe(int a, int b) {
8   if (a == b || b == 1) {
9
10
11   }
12   return 1;
13 }
14 return 2;
15 }
16 }
```

▼ Mutants
14:25, 28 Mar

Changing this 1 line to

```
if (a != b || b == 1) {
```

does not cause any test exercising them to fail.

Consider adding test cases that fail when the code is mutated to ensure those bugs would be caught.

Mutants ran because goranpetrovic is whitelisted

[Please fix](#) [Not useful](#)

Figure 1: Mutant finding shown in the Critique - Google code review tool

Regression Oracles

Generating Regression Oracles

First a bunch of mutants of the original program are generated (at they bytecode level)

Then, execute each test case against the mutant and record “*traces*” with necessary information to derive the assertion

After execution, the traces are analyzed for difference between runs. An assertion is added for each difference.

Generating Regression Oracles

Assertions are generated based on the output type of the Method Under Test

Primitive Assertions: `assertEquals(var, value);`

Where `var` is the output of MUT and `value` is the observed value on the non-mutated trace

Comparison Assertions: `assertTrue/False(var2.equals(var));`

Where `var` is the output of MUT and `var2` is an observed value of the same type on the non-mutated trace.

These assertions would fail if executed on a mutant

Generating Regression Oracles

Assertions are generated based on the output type of the Method Under Test

Inspector Assertions: `assertEquals(var2, value);`

Where `var2` is the output of **an observer method** and `value` is the observed value on the non-mutated trace

Field Assertions: `assertEquals(obj.VAR, value);`

Where `VAR` is a field on the object under test and `value` is an observed value of `VAR` on the non-mutated trace.

These assertions would fail if executed on a mutant

Generating Regression Oracles

Assertions are generated based on the output type of the Method Under Test

String Assertions: `assertEquals(var.toString(), value);`

Where `var` is the output of the Method Under Test and `value` is the observed value on the non-mutated trace

Not always useful as not all classes implement `toString`

Generating Regression Oracles

Given a unit under test P ,

Execute P and record values of program variables

$M = \text{genMutants}(P)$

For each mutant in M :

 Execute mutant and record values of program variables

 Add assertion which kills mutant based on type of MUT

Which approaches could generate oracles which find the bugs?

```
public static double calculatePrice(int basePrice, int daysUntilEvent, int customerAge, boolean isMember) {  
    double maxDiscount = basePrice - 100;  
  
    double discountRate = daysUntilEvent / maxDiscount; // BUG 1: potential division by zero  
  
    if (isMember) discountRate += .10;  
  
    if (customerAge >= 65 || customerAge < 18) {  
        discountRate += (customerAge - 65) / maxDiscount; //BUG 2: customerAge < 18 gives negative discount  
    }  
  
    return basePrice - (basePrice * discountRate);  
}
```

	Bug 1	Bug 2
Toradocu	☐	☐
IR-based approach	☐	☐
Neural approach	☐	☐
Regression oracle	☐	☐

Benchmarks for Evaluation

Defects4J

- Collection of reproducible bugs to evaluate testing
- 854 bugs from open source projects
- Includes a “patch” or fix
- Contains a “triggering test” that fails before the fix and passes after the fix

Source Code Version Control

The screenshot shows a GitHub commit page for commit `0209008` by user `dmcgrann`, committed on Feb 2, 2019. The commit message is "Automatically backed up by Learn". The commit is linked to a parent commit `bef722a`. The diff shows one file changed: `js/index.js`, with a total of +1 line and -1 line changed. The code diff for `js/index.js` is as follows:

```
@@ -34,7 +34,7 @@ function displayCommits() {
34 34     .map(
35 35     detail =>
36 36     '<li><strong>' +
37 - detail.username.login +
37 + detail.username.name +
38 38     '</strong> - ' +
39 39     detail.commit.message +
40 40     '</li>'
```

Source Code Version Control

- **Patch:** a set of changes that moves the code from the “before” version to the “after” version
 - Can involve modifying, adding, or removing lines of code
- **Checkout:** the process of switching between different versions of the code

Summary

- Mutation testing
- Regression oracles
 - reflect the current program behavior
 - generated by mutating P, executing, and observing the differences in program variables
- Defects4j benchmark